OghmaNotes

AI-Powered Note-Taking with RAG & Canvas LMS Integration

CT216 — Software Engineering I

Project Report

Academic Year 2025/2026

Semyon Fox	24495896
Samuel Regan	24498764
Shreyansh Singh	24316243

Lecturer: **Enda Barrett**

University of Galway

Live Application: https://oghmanotes.ie
GitHub Repository: https://github.com/semyonfox/oghma

April 2026

Contents

1 Project Overview **1**
1.1 Problem Statement . . . 1
1.2 Our Solution . . . 1
1.3 Key Differentiators . . . 1

2 Team Roles & Contributions **2**
2.1 Semyon Fox — Tech Lead / Full-Stack / DevOps . . . 2
2.2 Samuel Regan — Authentication / RAG Pipeline . . . 2
2.3 Shreyansh Singh — Infrastructure / OAuth / Canvas API . . . 2

3 Customer-Facing Features **3**
3.1 Landing Page . . . 3
3.2 Authentication . . . 4
3.2.1 Email/Password Authentication . . . 5
3.2.2 OAuth Authentication . . . 5
3.3 Notes Editor . . . 6
3.3.1 Editor Interface . . . 6
3.3.2 File Management . . . 6
3.4 Canvas LMS Integration . . . 7
3.4.1 Setup . . . 7
3.4.2 Import Process . . . 7
3.5 AI Chat (RAG-Powered) . . . 8
3.5.1 How It Works . . . 8
3.5.2 Context Scoping . . . 9
3.5.3 Technical Pipeline . . . 9
3.6 Settings . . . 10
3.7 Quiz System (Spaced Repetition) . . . 10
3.8 Productivity Tools . . . 10
3.9 Internationalisation . . . 11

4 Technical Architecture **11**
4.1 System Overview . . . 11
4.2 Technology Stack . . . 12
4.3 AWS Infrastructure . . . 12

5 REST API **13**
5.1 Authentication Middleware . . . 13

6 Key System Flows **14**
6.1 Authentication Flow . . . 14
6.2 Canvas Import Pipeline . . . 15
6.3 RAG Query Flow . . . 16

7 Database Design **17**
7.1 Schema Overview . . . 17
7.2 Core Tables . . . 17
7.3 Vector Storage Design . . . 18

7.4 Migration History . . . 18

8 CI/CD & Deployment **18**
8.1 AWS Amplify Pipeline . . . 18
8.2 Branch Strategy . . . 18
8.3 ECS Fargate Deployment . . . 19

9 Design Decisions & Architectural Pivots **19**
9.1 Pivot 1: Notea Fork to Custom Next.js Application . . . 19
9.2 Pivot 2: Monorepo to Single Application . . . 19
9.3 Pivot 3: BullMQ (Redis) to AWS SQS . . . 20
9.4 Pivot 4: Local Ollama to Cohere Embeddings . . . 20
9.5 Pivot 5: OpenWebUI Auth Layer to Direct LLM API . . . 20
9.6 Pivot 6: pnpm to npm . . . 20
9.7 Pivot 7: Cohere Embeddings Back to Self-Hosted Homelab . . . 20
9.8 Budget Constraints . . . 21

10 Testing **21**
10.1 Testing Approach . . . 21
10.2 What Was Tested . . . 21
10.3 Honest Assessment . . . 21

11 Project Management **21**
11.1 Methodology . . . 22
11.2 GitHub Projects . . . 22
11.3 Timeline . . . 23
11.4 Commit Activity . . . 23

12 Challenges & Reflections **23**
12.1 Semyon Fox . . . 23
12.2 Samuel Regan . . . 24
12.3 Shreyansh Singh . . . 25

13 Future Work **25**

14 AI-Assisted Development Disclosure **25**
14.1 How AI Was Used . . . 26
14.2 What AI Did Not Do . . . 26
14.3 Reflection . . . 26

15 Resources & Links **27**

1 Project Overview

1.1 Problem Statement

University students face a persistent problem with managing lecture materials. Course content is scattered across Learning Management Systems (LMS), local folders, and personal notes. Existing note-taking tools such as Notion or Obsidian offer strong editors, but none combine a notes workspace with AI-powered search over lecture PDFs, nor do they integrate directly with university platforms like Canvas LMS.

Students frequently need to search through dozens of lecture PDFs to find a specific concept, cross-reference notes with slides, or quickly locate an assignment brief buried in a Canvas module. Current workflows involve manually downloading files, switching between applications, and relying on basic keyword search — which fails when the terminology in notes differs from the source material.

The market for student productivity tools is crowded yet underserved in a critical dimension. Notion, Obsidian, and Google Docs offer strong editing experiences but treat uploaded documents as opaque files — users cannot query their content semantically. Dedicated AI tools like ChatGPT or Perplexity can answer questions but lack persistent knowledge bases scoped to a student's own course materials. No existing tool bridges the gap between a personal note-taking workspace and AI-powered retrieval over university lecture content, integrated with the institution's own LMS.

1.2 Our Solution

OghmaNotes is an AI-powered note-taking platform purpose-built for students. It provides a unified workspace where users can create and organise notes using a VSCode-inspired editor, import course materials directly from Canvas LMS, and query their entire knowledge base through a Retrieval-Augmented Generation (RAG) chat interface.

The platform is built with Next.js 16.2 on the frontend, a PostgreSQL database with the `pgvector` extension for vector similarity search, and a suite of AWS services (RDS, S3, SQS, ECS Fargate, SES, Amplify) for production-grade infrastructure. It is live at `https://oghmanotes.ie`.

1.3 Key Differentiators

OghmaNotes distinguishes itself from existing tools in several ways. First, RAG-powered search allows users to ask natural-language questions about their notes and uploaded PDFs, with the system retrieving semantically relevant passages rather than relying on exact keyword matches. Second, direct Canvas LMS integration enables one-click import of courses, modules, files, and assignments, processed in the background by a self-scaling worker. Third, the platform runs on a full AWS production stack, demonstrating real-world cloud engineering rather than managed PaaS solutions. The project involved over 807 commits across 65 active development days, with the application supporting 12 languages through a custom internationalisation system.

2 Team Roles & Contributions

The team comprised three members, each responsible for distinct areas of the project. Over seven months of development (September 2025 – April 2026), the team collectively produced over 807 commits across 65 active development days.

2.1 Semyon Fox — Tech Lead / Full-Stack / DevOps

Served as technical lead responsible for system architecture, infrastructure decisions, and the majority of feature development. Key contributions include:

- **Architecture & Infrastructure:** Designed and implemented AWS infrastructure including Amplify hosting, SQS job queues, ECS Fargate workers, and ElastiCache. Made all major architectural decisions including the BullMQ to SQS migration and embedding pipeline iterations.
- **Frontend:** Built the VSCode-style note editor UI, file tree, command palette, settings page, Canvas import UI, and AI chat interface. Implemented internationalisation across 12 languages.
- **Backend:** Developed API routes for notes CRUD, file upload, AI chat with RAG, and auth middleware with rate limiting.
- **DevOps:** Configured Amplify CI/CD pipeline, ECS Fargate worker deployment, and environment management across dev/main/production branches.
- **Canvas Integration:** Built the full Canvas LMS import pipeline from API integration through to S3 storage and RAG embedding.

2.2 Samuel Regan — Authentication / RAG Pipeline

Contributed to the authentication system, RAG pipeline, and frontend:

- **Authentication:** Built the login and registration UI, implemented password reset with Nodemailer (later migrated to AWS SES), and developed JWT-based session tokens with middleware-based route protection across the application.
- **Database:** Designed the original schema for the embeddings and chunks tables, later extended this to track worker state.
- **RAG Pipeline:** Implemented the full ingestion flow including PDF text extraction, text chunking, embedding generation, and pgvector similarity search. Designed an asynchronous job queue with a PostgreSQL jobs table to separate document ingestion from the upload request to avoid without timeout failures on large imports. The chat prompt construction and HNSW indexing were later extended by the rest of the team.
- **Frontend:** Contributed targeted fixes to the file rename and zoom functionality.

2.3 Shreyansh Singh — Infrastructure / OAuth / Canvas API

Contributed to the cloud infrastructure setup, authentication, and Canvas LMS integration:

- **Infrastructure:** Designed and provisioned core AWS services including RDS (PostgreSQL), S3 (object storage), and SES (transactional email). Configured security groups, IAM policies, and SSL certificates for database connectivity.
- **Database:** Designed the PostgreSQL schema under the `app` namespace (e.g. `app.login`, `app.notes`, `app.chunks`), making decisions around table structure, relationships, and the use of `pgvector` for embedding storage.
- **OAuth:** Implemented Google and GitHub OAuth flows using Auth.js (NextAuth), including provider registration, callback handling, and account linking logic.
- **Canvas API Integration:** Integrated the Canvas LMS REST API into the system, enabling course and file retrieval from university Canvas instances.

3 Customer-Facing Features

This section documents the major features of OghmaNotes as experienced by users. Each subsection describes the feature's purpose, implementation approach, and provides screenshots of the interface.

3.1 Landing Page

The public-facing landing page at `https://oghmanotes.ie` serves as the marketing site for OghmaNotes. It features a hero section with the tagline and call-to-action, a features overview with four key selling points (AI-Powered Insights, Seamless Canvas Integration, Smart Organisation, Work Anywhere), an about section with team information, and a footer with university partner logos.

The landing page was built using Next.js App Router with server-side rendering, styled with Tailwind CSS v4. The navigation bar provides links to Features, About, Blog, and Contact pages, along with authentication entry points.

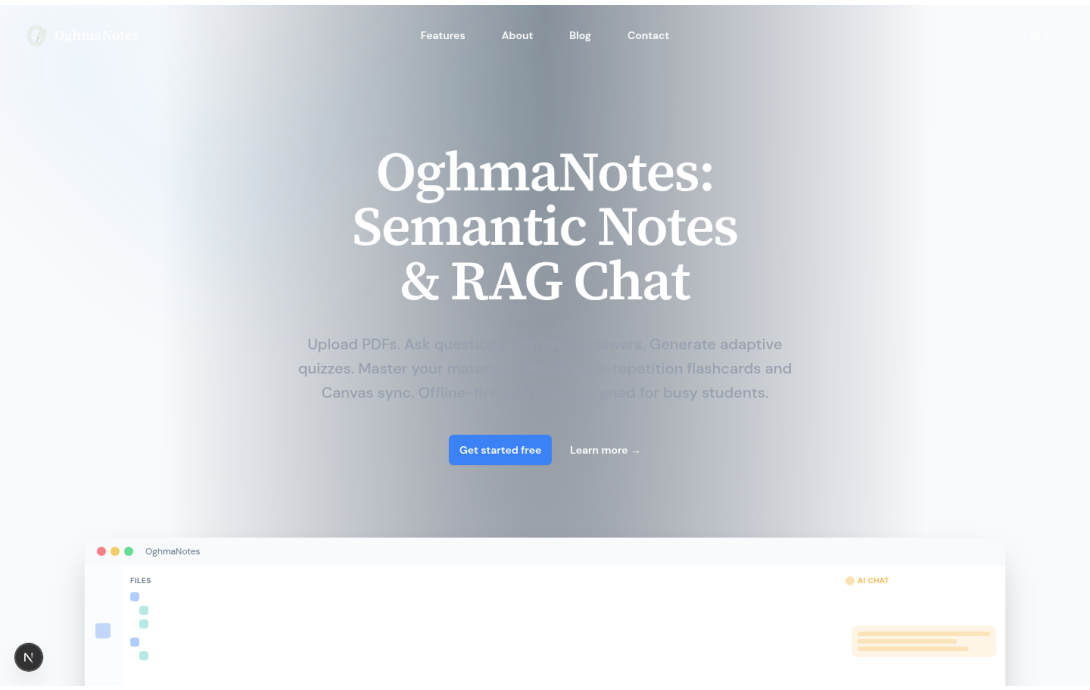

Figure 1: Landing Page — Hero section with tagline and call-to-action

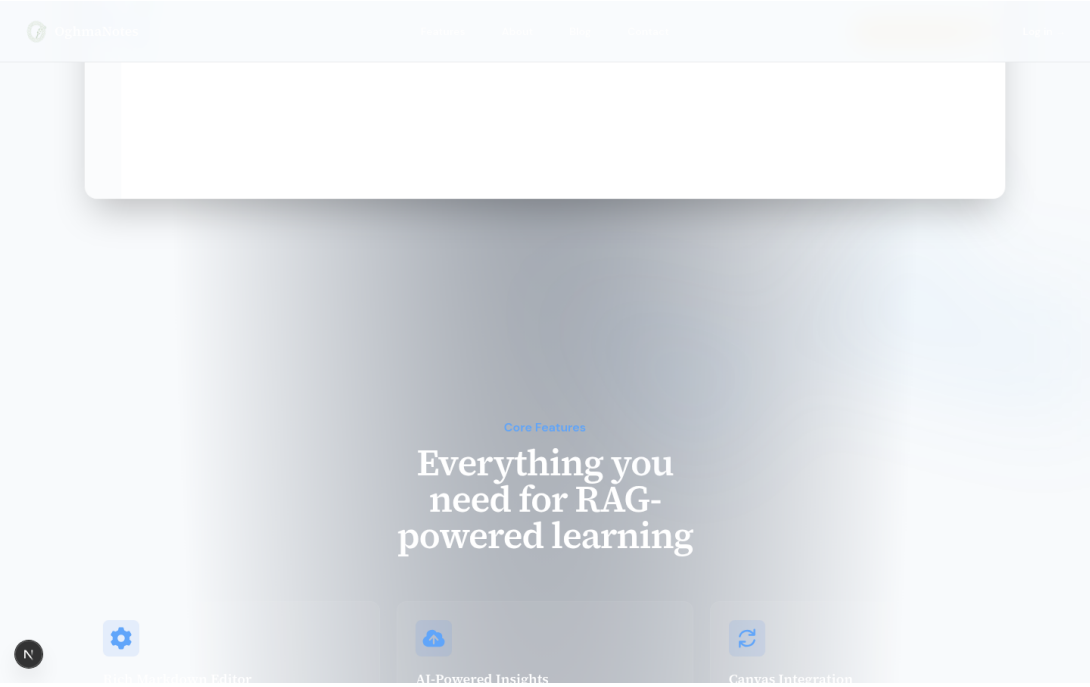

Figure 2: Landing Page — Core feature cards

3.2 Authentication

OghmaNotes supports two authentication paths: traditional email/password registration and OAuth via third-party providers.

3.2.1 Email/Password Authentication

Users can create an account with an email address and password. The registration flow includes email verification via AWS SES, with a verification link sent to the user's inbox. Login uses JWT-based sessions with secure, HTTP-only cookies. Password reset is available via a token-based email flow, also powered by SES.

3.2.2 OAuth Authentication

The application supports OAuth 2.0 login via Google and GitHub, implemented using Auth.js (the successor to NextAuth.js). Users who register with email/password can subsequently link OAuth providers to their account from the settings page. The OAuth implementation handles account deduplication — if a user signs in with Google using the same email as an existing account, the accounts are linked automatically.

Rate limiting is applied to all authentication endpoints to prevent brute-force attacks.

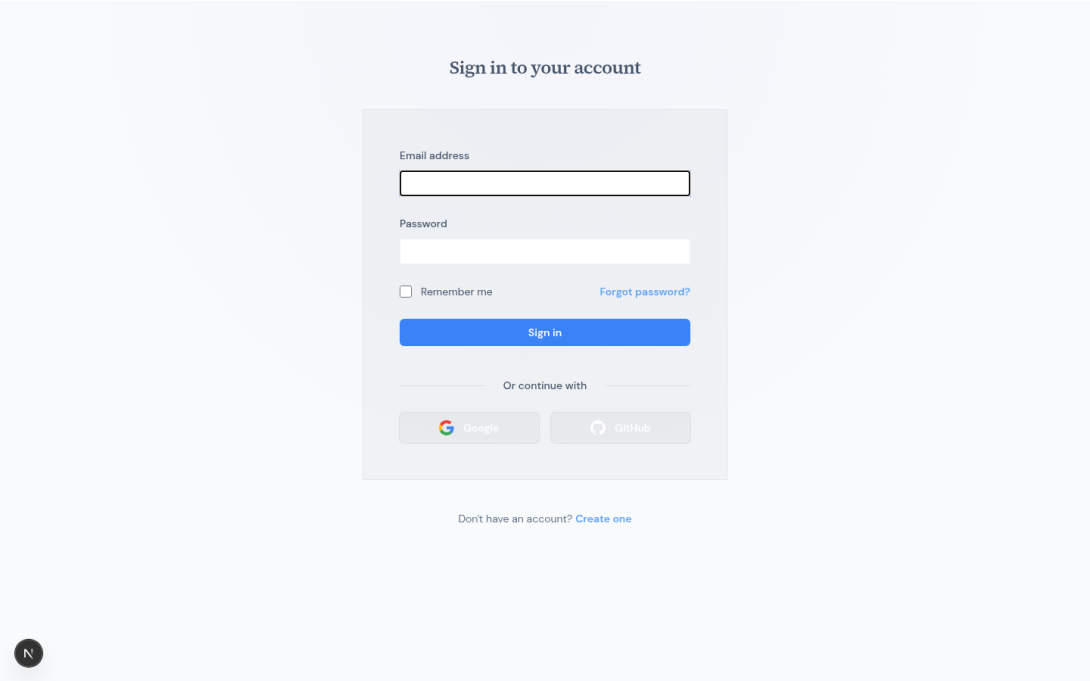

Figure 3: Login Page — Email/password form with Google and GitHub OAuth

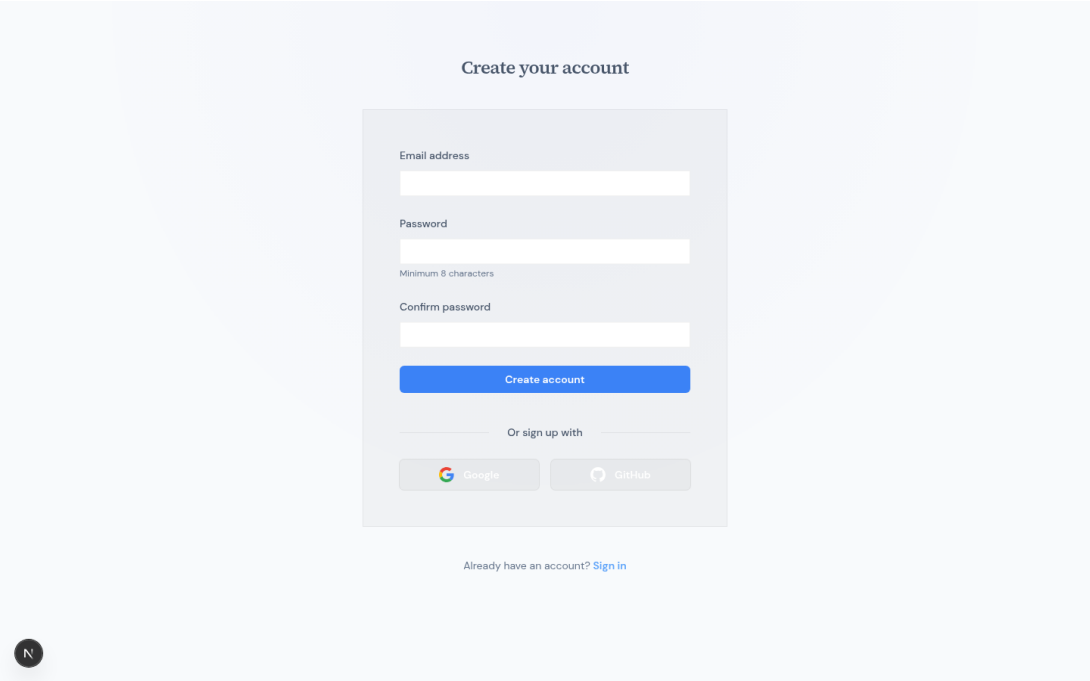

Figure 4: Registration Page — Account creation form with OAuth options

3.3 Notes Editor

The core of OghmaNotes is a VSCode-inspired note editor with a three-panel layout.

3.3.1 Editor Interface

The editor features a **file tree sidebar** on the left, styled after Obsidian with indent guides and compact rows, providing hierarchical navigation through folders and files. The **main editor pane** uses a rich-text editor built on the Lexical framework with CodeMirror for markdown source editing, supporting markdown-style formatting, code blocks with syntax highlighting, and inline media. A **split-pane view** allows users to open a PDF viewer alongside their notes, enabling side-by-side study of lecture materials and note-taking.

3.3.2 File Management

Users can create, rename, delete, and duplicate both files and folders. Drag-and-drop file upload is supported for PDFs and images. A breadcrumb navigation bar shows the current file's location in the hierarchy, and a command palette (accessible via keyboard shortcut) provides quick access to common actions.

Screenshot: Notes Editor — File tree sidebar with split-pane editor and PDF viewer

Figure 5: Notes Editor — File tree sidebar with split-pane editor and PDF viewer

Screenshot: Notes Editor — Command palette overlay

Figure 6: Notes Editor — Command palette overlay

3.4 Canvas LMS Integration

One of OghmaNotes' key differentiators is direct integration with Canvas LMS, the learning management system used by many universities.

3.4.1 Setup

Users provide their Canvas API token and institution URL in the settings page. The token is encrypted and stored as the `canvas_token` column on the `app.login` table, alongside the `canvas_domain`. The encrypted token is decrypted at runtime to authenticate API requests to the Canvas REST API.

3.4.2 Import Process

Once connected, users can browse their Canvas courses and select which ones to import. The import process runs asynchronously:

1. The frontend sends an import request to the API, which creates a job record in PostgreSQL and publishes a message to AWS SQS.
2. An ECS Fargate worker picks up the SQS message and begins processing.
3. The worker retrieves courses, modules, files, and assignments from the Canvas API.
4. Downloaded files are uploaded to S3, with metadata stored in PostgreSQL.

5. PDFs undergo OCR/text extraction (via Marker API with `pdf-parse` fallback), chunking, and embedding generation.
6. Embeddings are stored in pgvector for RAG search.
7. The worker self-scales to zero after an idle period, minimising costs.

Real-time progress is communicated back to the user through status polling and toast notifications.

Screenshot: Canvas Import — Settings page with API token configuration

Figure 7: Canvas Import — Settings page with API token configuration

Screenshot: Canvas Import — Course selection and import progress

Figure 8: Canvas Import — Course selection and import progress

3.5 AI Chat (RAG-Powered)

The AI chat interface allows users to ask natural-language questions about their notes and imported documents.

3.5.1 How It Works

When a user sends a message, the system generates an embedding of the query using a self-hosted embedding model (4096 dimensions, served via the project's homelab infrastructure). This embedding is compared against stored document chunks using pgvector's cosine similarity search. The top-k most relevant chunks are retrieved and injected into a prompt, which is sent to the LLM (Kimi K2.5 via Moonshot AI). The response is streamed back to the chat interface.

3.5.2 Context Scoping

Users can attach specific files or folders to a chat session to scope the RAG search. This ensures that when studying for a particular module, only materials from that module are included in search results — improving relevance and reducing noise from unrelated content.

Chat sessions persist across visits, allowing users to return to previous conversations.

3.5.3 Technical Pipeline

The RAG pipeline combines several components to deliver accurate, source-grounded answers. User queries are embedded using a self-hosted `qwen3-embedding:8b` model producing 4096-dimensional vectors, served from the team's homelab via a Cloudflare Tunnel. The embedding service uses asymmetric input types: `search_document` for indexing and `search_query` for queries, which improves retrieval quality by accounting for the stylistic difference between documents and questions.

Before embedding, all markdown syntax is stripped from note content — headers, emphasis markers, links, code fences, and HTML tags are removed to prevent formatting tokens from diluting the semantic signal in the vector space. Text is then split into sentence-aligned chunks of approximately 500 characters with no overlap, splitting on sentence boundaries (`/(?<=[.?!])\s+/`).

At query time, pgvector performs exact cosine similarity search using the `<=>` operator, retrieving up to 20 candidates within a maximum distance of 0.75. At 4096 dimensions, the full sequential scan trades index efficiency for higher-fidelity vector comparisons — acceptable at current data volumes. These candidates are then reranked using a self-hosted reranker model, which applies cross-attention scoring to the query–document pairs. Chunks scoring below a 0.15 relevance threshold are discarded, and the top 5 are retained.

The surviving chunks are grouped by source note in the prompt, with a 1200-character limit per note block, and the LLM is instructed to cite sources by note title. The production LLM is Kimi K2.5 (Moonshot AI) accessed via an OpenAI-compatible API, configured with a 2048-token generation limit. Responses are streamed to the client via server-sent events with idle timeout reset to maintain connection liveness.

Screenshot: AI Chat — Chat interface with RAG-powered responses and context attachments

Figure 9: AI Chat — Chat interface with RAG-powered responses and context attachments

3.6 Settings

The settings page provides account management and application configuration:

- **Profile Management:** Update display name, email, and avatar.
- **Theme:** Toggle between dark and light modes, implemented with class-based Tailwind v4 theming.
- **Language:** Select from 13 supported locales, powered by a custom internationalisation system.
- **Canvas Integration:** Configure Canvas API token and institution URL.
- **OAuth Accounts:** View and manage linked OAuth providers (link/unlink Google and GitHub).

Screenshot: Settings Page — Profile, theme, language, and Canvas configuration

Figure 10: Settings Page — Profile, theme, language, and Canvas configuration

3.7 Quiz System (Spaced Repetition)

OghmaNotes includes a flashcard-based quiz system that uses the FSRS (Free Spaced Repetition Scheduler) algorithm via the `ts-fsrs` library. Users can create flashcards from their notes and study them with scientifically-backed scheduling that optimises long-term retention. The system tracks quiz sessions, individual card reviews, and maintains streak data to encourage consistent study habits.

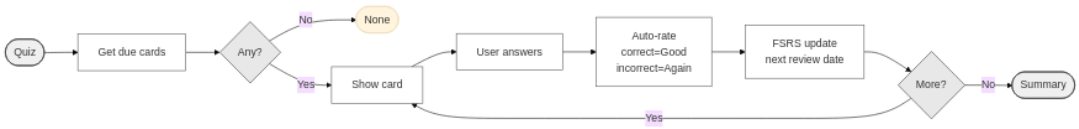

Figure 11: Quiz flow: FSRS card scheduling, difficulty rating, and interval update

3.8 Productivity Tools

The application includes several productivity features to support focused study:

- **Pomodoro Timer:** A built-in Pomodoro timer for structured study sessions, with session tracking stored in the database.

- **Time Blocks:** A time-blocking system that allows users to schedule dedicated study periods, viewable in a calendar week view.
- **PDF Annotations:** Users can annotate PDF documents directly within the viewer, with annotations persisted to the database.
- **Assignment Tracking:** Canvas assignments are synced and tracked, with due dates displayed in the calendar view.

Screenshot: Productivity Tools — Pomodoro timer, time blocks, and calendar view

Figure 12: Productivity Tools — Pomodoro timer, time blocks, and calendar view

3.9 Internationalisation

OghmaNotes supports 12 languages: English, Arabic, German, Spanish, French, Irish, Hindi, Italian, Dutch, Russian, Swedish, and Chinese (Simplified). The i18n system uses a custom locale framework with JSON translation files, where the active locale is stored in user preferences and applied across all UI strings. An automated translation pipeline was used to generate initial translations, which were then reviewed for accuracy.

4 Technical Architecture

4.1 System Overview

OghmaNotes follows a serverless-first architecture deployed entirely on AWS. The application consists of three main runtime components: the Next.js frontend and API (hosted on AWS Amplify), the PostgreSQL database (AWS RDS), and the Canvas import worker (AWS ECS Fargate). These are connected through a set of AWS services that handle storage (S3), messaging (SQS), caching (ElastiCache), and email (SES).

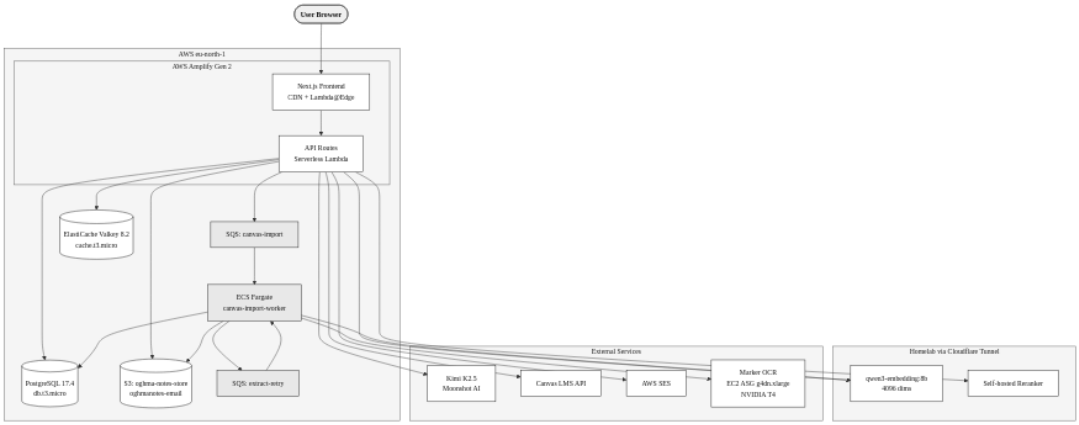

Figure 13: System Architecture — AWS infrastructure, homelab inference services, and external API dependencies

4.2 Technology Stack

Layer	Technology	Notes
Frontend	Next.js 16.2, React 19, TypeScript, Tailwind v4	VSCode-inspired split-pa
Backend	Next.js API Routes (serverless)	REST pattern with route
Database	PostgreSQL 17.4 + pgvector (AWS RDS)	`app.*` schema, 8+ migrat
Object Storage	AWS S3	Notes, attachments, Canv
Cache	AWS ElastiCache (Valkey 8.2)	Session cache
Job Queue	AWS SQS	Canvas import job dispat
Worker	AWS ECS Fargate	Self-scaling import worke
Email	AWS SES	Password reset, notificati
CI/CD	AWS Amplify Gen 2	Build, deploy, CDN, SSR
Auth	NextAuth v5 + Custom JWT	OAuth (Google/GitHub)
Embeddings	`qwen3-embedding:8b` (homelab, 4096 dims)	Self-hosted via Ollama; C
Reranking	Self-hosted (Cohere fallback)	Purpose-built relevance s
LLM	Kimi K2.5 (Moonshot AI)	Production LLM via Ope
OCR	Marker (self-hosted EC2 ASG, g4dn.xlarge) + Datalab fallback	PDF/DOCX/PPTX text
GPU Compute	AWS EC2 (g4dn.xlarge, NVIDIA T4, spot ~$0.35/hr)	Marker OCR via ASG, sc
LMS	Canvas LMS REST API	Course/file import pipeli
i18n	Custom locale system	12 languages

Table 1: OghmaNotes Technology Stack

4.3 AWS Infrastructure

The production infrastructure runs in the `eu-north-1` (Stockholm) region. The key services and their roles are:

AWS Amplify Gen 2 hosts the Next.js application, providing automatic builds from GitHub pushes, CDN distribution, and server-side rendering via Lambda@Edge. The build pipeline runs on each push to the `main` branch, executing `npm ci`, `next build`, and deploying the output.

AWS RDS runs PostgreSQL 17.4 with the `pgvector` extension enabled. The database uses a `db.t3.micro` instance with SSL-encrypted connections. All application tables live under the `app` schema to separate them from system tables.

AWS S3 stores all user-uploaded files, notes content, and Canvas-imported documents. Files are organised by user ID and note ID, with presigned URLs used for secure client-side uploads and downloads.

AWS SQS serves as the message queue for Canvas import jobs. When a user initiates an import, the API publishes a message to the SQS queue containing the job ID and user credentials. This decouples the request-handling latency from the potentially long-running import process.

AWS ECS Fargate runs the Canvas import worker as a containerised service. The worker polls SQS for messages, processes imports (downloading files from Canvas, uploading to S3, running OCR, generating embeddings), and scales to zero when idle to minimise costs.

AWS SES handles transactional email for password reset flows and email verification.

AWS ElastiCache (Valkey 8.2) provides in-memory caching for session data.

AWS EC2 runs a `g4dn.xlarge` GPU instance (NVIDIA T4) hosting the Marker OCR server for document text extraction. The instance runs within an Auto Scaling Group (ASG) behind an Application Load Balancer, using spot pricing at approximately $0.35/hour. The ASG scales to zero when idle, with a 4–7 minute cold start when documents need processing. The instance supports PDF, DOCX, DOC, PPTX, PPT, PNG, and JPG formats. If the EC2 instance is unavailable, the system falls back to the Datalab Convert API as an external OCR service.

5 REST API

The OghmaNotes backend exposes a REST API through Next.js API Routes, with all endpoints prefixed under `/api/`. The API comprises 16 route groups covering authentication (registration, login, OAuth callbacks, password reset), notes CRUD, file tree operations, S3-backed file upload, Canvas connection and import management, AI chat with session persistence, vector and full-text search, PDF extraction, user settings, quiz/flashcard management, Pomodoro and time-block tracking, Canvas assignment sync, and data portability (vault export/import). A health check endpoint is also exposed for monitoring.

5.1 Authentication Middleware

All protected routes use a custom JWT middleware that extracts the token from HTTP-only cookies, verifies the signature, and attaches the decoded user object to the request context. Unauthenticated requests receive a 401 response. Rate limiting is implemented using a sliding-window counter pattern, with stricter limits on authentication endpoints (login, register, password reset) to prevent brute-force attacks.

6 Key System Flows

This section presents the critical operational flows within OghmaNotes, illustrating how components interact to deliver key features.

6.1 Authentication Flow

The authentication system supports two parallel paths that converge on a shared JWT session mechanism.

Email/Password Path: The user submits credentials to `POST /api/auth/register`, which hashes the password with bcrypt, creates a user record in PostgreSQL, and sends a verification email via SES. Upon email verification, the user can log in via `POST /api/auth/login`, which validates credentials, generates a JWT, and sets it as an HTTP-only cookie.

OAuth Path: The user clicks an OAuth provider button (Google and GitHub), which redirects to the provider's consent screen via Auth.js. On callback, Auth.js verifies the token, checks for an existing account with the same email, and either creates a new user or links the OAuth account to an existing one. A JWT is issued identically to the email/password path.

Token Refresh: JWTs have a configurable expiry. The `/api/auth/me` endpoint checks token validity and issues a refreshed token when needed.

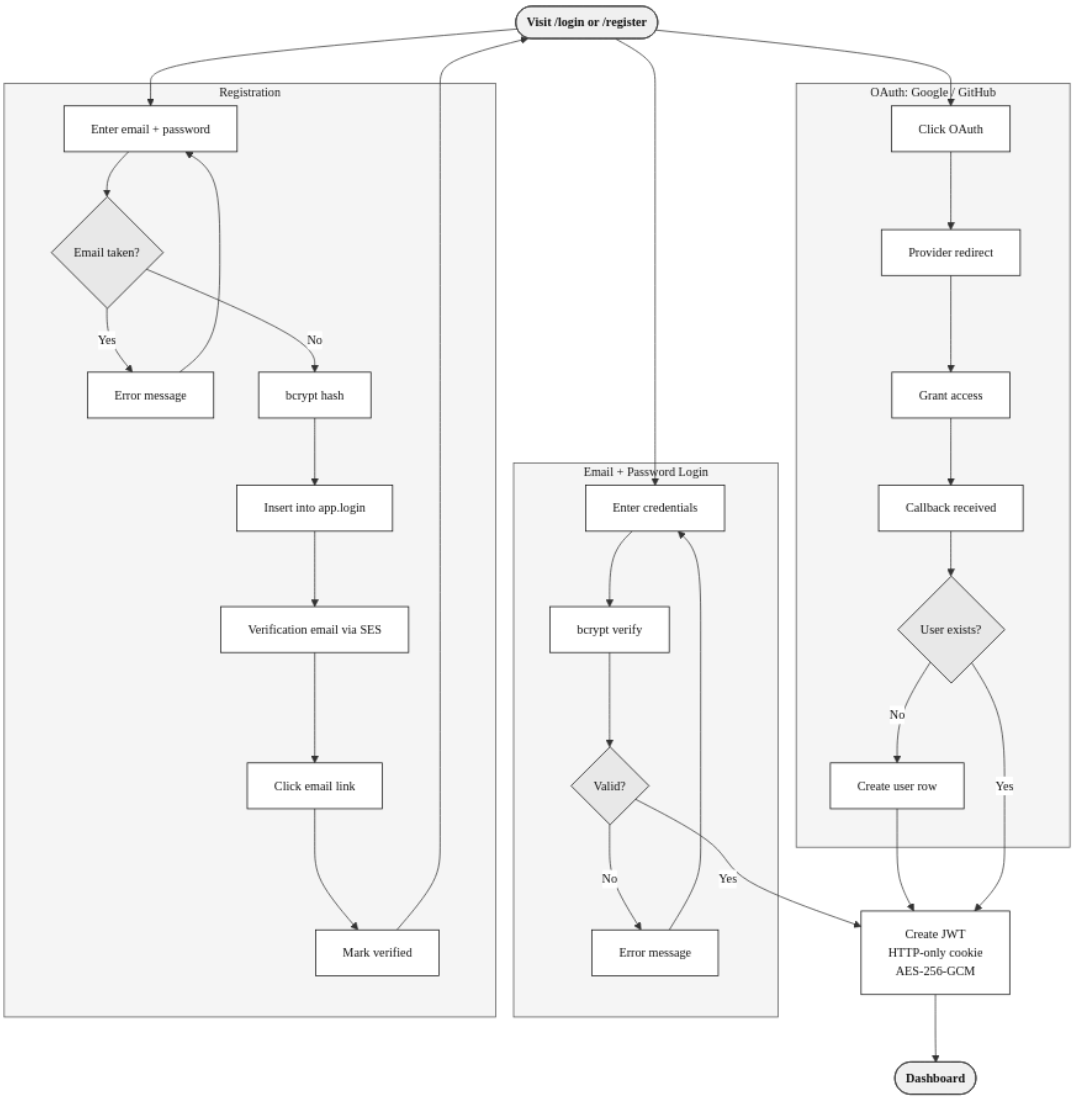

Figure 14: Authentication Flow — Email/password, registration with email verification, and OAuth paths

6.2 Canvas Import Pipeline

The Canvas import pipeline is the most architecturally complex flow in the system, spanning five AWS services.

1. **Trigger:** User clicks "Import" in the Canvas settings UI. The frontend calls `POST /api/canvas/import` with selected course IDs.
2. **Job Creation:** The API creates a `canvas_import_jobs` record in PostgreSQL (status: `queued`) and publishes a JSON message to the SQS queue containing the job ID and encrypted Canvas token.
3. **Worker Activation:** The ECS Fargate worker, which polls SQS on a loop, receives the message and begins processing.
4. **Canvas API Calls:** The worker uses the Canvas REST API to enumerate courses, modules, files, and assignments. Each file is downloaded to a temporary buffer.

5. **S3 Upload:** Downloaded files are uploaded to S3 under a user-specific key prefix, with metadata recorded in the `canvas_imports` table.
6. **Text Extraction:** PDFs (and DOCX/PPTX files) are sent to a self-hosted Marker instance running on an EC2 `g4dn.xlarge` GPU instance (NVIDIA T4) within an Auto Scaling Group. If the EC2 instance is unavailable, the system falls back to the Datalab Convert API. Markdown syntax is stripped before embedding to reduce noise in the vector space.
7. **Chunking & Embedding:** Extracted text is split into sentence-aligned, non-overlapping chunks of approximately 500 characters. Each chunk is sent to the self-hosted embedding service, producing a 4096-dimensional vector.
8. **Vector Storage:** Embeddings are stored in the `note_chunks` table using pgvector with cosine distance search.
9. **Completion:** The job status is updated to `complete` (or `failed` with error details). The worker returns to polling and scales to zero after an idle timeout.

Figure 15: Canvas Import Pipeline — SQS job dispatch, ECS Fargate worker processing, and pgvector storage

6.3 RAG Query Flow

When a user sends a message in the AI chat interface, the following pipeline executes:

1. **Query Embedding:** The user's message is sent to the self-hosted embedding service, producing a 4096-dimensional query vector.
2. **Vector Search:** The query vector is compared against stored chunk embeddings using pgvector's `<=>` (cosine distance) operator. The top-k most similar chunks are retrieved (filtered to a maximum cosine distance of 0.75), optionally scoped to the user's selected context (specific files or folders).
3. **Reranking:** Retrieved chunks are passed through a self-hosted reranker model, which rescores them by relevance to the original query. Chunks below a minimum relevance threshold (0.15) are discarded. This step significantly improves precision over raw vector similarity alone.
4. **Prompt Construction:** The reranked chunks are injected into a system prompt that instructs the LLM to answer based on the provided context, cite sources, and indicate when information is not available.
5. **LLM Generation:** The prompt is sent to the LLM (Kimi K2.5 via Moonshot AI's OpenAI-compatible API). The response is streamed back to the client using server-sent events for a responsive user experience.
6. **Session Persistence:** The message and response are stored in the `chat_messages` table, linked to the user's chat session for future reference.

The pipeline includes several optimisation decisions driven by empirical testing. The 20-candidate retrieval with reranking to 5 was chosen because raw vector similarity alone produces false positives — semantically adjacent but topically irrelevant chunks. The self-hosted reranker provides a cross-attention score that considers the query–document pair holistically, significantly improving precision. The 0.75 cosine distance threshold and 0.15 relevance floor were tuned against a test set of university lecture queries to balance recall and noise. Markdown syntax is stripped before embedding (removing headers, emphasis markers, code fences, and HTML tags) because formatting tokens dilute the semantic signal in the vector space.

Figure 16: RAG Query Pipeline — Query embedding, pgvector cosine search, self-hosted reranking, and LLM streaming

7 Database Design

7.1 Schema Overview

The database uses PostgreSQL 17.4 with the `pgvector` extension, hosted on AWS RDS. All application tables reside under the `app` schema, cleanly separating application data from PostgreSQL system tables. The schema evolved over 8+ migrations, reflecting an iterative development approach.

7.2 Core Tables

The schema contains 19 tables under the `app` namespace, organised around a user-centric model. The primary tables are `app.login` (user accounts and credentials), `app.notes` and `app.tree_items` (note content and hierarchical file structure), `app.chunks` and `app.embeddings` (RAG text chunks and their 4096-dimensional vectors), and `app.chat_sessions`/`app.` (AI conversation persistence). Supporting tables handle OAuth account linking, Canvas import job tracking, quiz flashcards with FSRS scheduling, Pomodoro sessions, time blocks, PDF annotations, and rate limiting. A `schema_migrations` table tracks the 8+ migrations applied over the project lifetime.

Screenshot: Entity-Relationship Diagram — Database schema relationships

Figure 17: Entity-Relationship Diagram — Database schema relationships

7.3 Vector Storage Design

The `app.chunks` and `app.embeddings` tables are central to the RAG system. Each chunk row contains the text, its source document ID, and user ownership. A corresponding row in `app.embeddings` stores the 4096-dimensional vector embedding, linked via `chunk_id`. Similarity search uses pgvector's `<=>` cosine distance operator with a full sequential scan.

The embedding pipeline iterated through multiple providers before settling on the current configuration (see Section 9). The production setup uses `qwen3-embedding:8b` at full 4096 dimensions for maximum semantic fidelity. While 4096 dimensions exceeds pgvector's HNSW index limit of 2000 and precludes approximate nearest-neighbour indexing, the sequential scan is acceptable at current data volumes and preserves retrieval quality.

7.4 Migration History

The database evolved through multiple SQL migrations, each tracked in the `app.schema_migrations` table. Notable migrations include the initial schema creation (`001`), the addition of OAuth accounts (`002`), quiz session card IDs (`003`), vault job columns (`005`), Canvas import uniqueness constraints (`006`), and email verification support. This iterative migration approach reflects agile development practices, with the schema adapting to new requirements as features were built.

8 CI/CD & Deployment

8.1 AWS Amplify Pipeline

The project uses AWS Amplify Gen 2 as the primary CI/CD platform. The build and deployment pipeline is triggered automatically on each push to the `main` branch. The pipeline executes the following stages:

1. **Install:** Runs `npm ci` to install dependencies from the lockfile.
2. **Build:** Executes `next build`, producing optimised static assets and serverless functions.
3. **Deploy:** Distributes the build output to Amplify's CDN and deploys SSR routes as Lambda@Edge functions.

Environment variables (database credentials, API keys, AWS service endpoints) are managed through Amplify's environment configuration, with separate variable sets for the `dev` and `main` branches.

8.2 Branch Strategy

The repository follows a two-branch model. The `dev` branch serves as the integration branch where features are developed and tested. When stable, changes are merged to `main` via pull request, triggering the production deployment. Both branches have corresponding Amplify environments with independent configurations.

8.3 ECS Fargate Deployment

The Canvas import worker runs as a separate ECS Fargate service. Its container image is built from a Dockerfile in the repository and pushed to Amazon ECR. The ECS task definition specifies resource limits, environment variables, and the SQS queue URL. The service is configured with a desired count of zero by default, scaling up when SQS messages are detected.

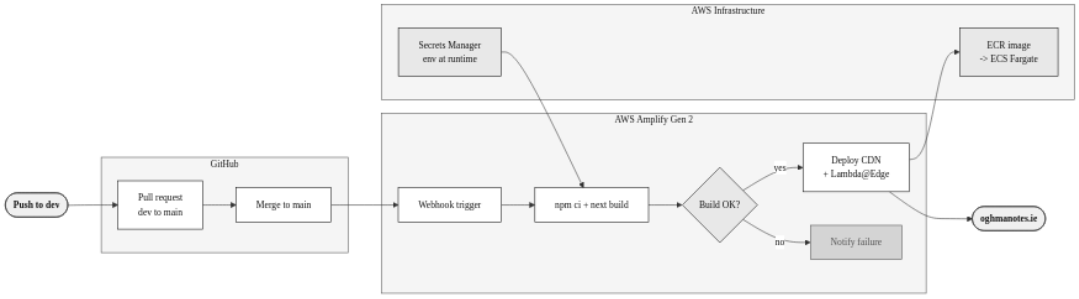

Figure 18: CI/CD Pipeline — GitHub push to AWS Amplify build, deploy, and infrastructure update

9 Design Decisions & Architectural Pivots

Over the course of development, the team made several significant architectural pivots. Each represents a real engineering decision driven by constraints encountered during implementation. These pivots are documented here as evidence of iterative design and practical problem-solving.

9.1 Pivot 1: Notea Fork to Custom Next.js Application

The project initially began with Notea, an open-source note-taking application, as a starting point. The intent was to extend it with RAG capabilities and Canvas integration. However, Notea's architecture did not support the vector search, LMS integration, or AWS infrastructure required by OghmaNotes. The team ultimately stripped Notea down to its core and rebuilt most components, migrating from the fork to a clean Next.js application.

Lesson: Forking an existing project can save time initially but creates technical debt when the original architecture diverges significantly from the target design. Starting fresh would have been faster in retrospect.

9.2 Pivot 2: Monorepo to Single Application

The project started as a monorepo with separate packages under an `apps/web/` directory. This introduced complexity in the Amplify build configuration that proved disproportionate to the benefits for a three-person team. The team collapsed the structure into a single Next.js application, simplifying both development and deployment.

9.3 Pivot 3: BullMQ (Redis) to AWS SQS

The Canvas import job queue was initially implemented using BullMQ backed by ElastiCache (Valkey/Redis). This worked in local development but failed in production: Amplify's Lambda functions could not reach ElastiCache without VPC configuration, and the Amplify Lambda runtime had a 29-second timeout that was incompatible with long-running import jobs. The team migrated to AWS SQS, which uses IAM authentication (no VPC required) and decouples the API from the worker entirely.

Lesson: Cloud networking constraints drive architectural decisions more than theoretical best practices. SQS was the pragmatic choice given the deployment environment.

9.4 Pivot 4: Local Ollama to Cohere Embeddings

The initial embedding pipeline used a self-hosted Ollama instance running `qwen3-embedding:8b`, served over a Cloudflare Tunnel. While the model itself performed well, the Cloudflare Tunnel introduced network-level instability — embedding requests during Canvas imports would intermittently time out, causing ingestion jobs to fail mid-batch. Rather than debug the tunnel reliability, the team migrated to Cohere's `embed-multilingual-v3.0` API, which provided a stable external endpoint with consistent latency. As a secondary benefit, Cohere's 1024-dimensional output was also compatible with pgvector's HNSW index limit.

Lesson: Infrastructure reliability is as important as model quality. A locally-hosted model routed through a tunnelled connection introduces failure modes that a direct API call does not.

9.5 Pivot 5: OpenWebUI Auth Layer to Direct LLM API

The RAG pipeline initially routed LLM requests through OpenWebUI as an authentication intermediary. This added latency and complexity without significant benefit. The team removed OpenWebUI and implemented direct API calls to the LLM provider, with a dual-provider pattern that switches between Ollama (development) and Kimi K2.5 (production) via environment variables.

9.6 Pivot 6: pnpm to npm

A mid-project package manager switch from pnpm to npm was necessitated by compatibility issues with the Amplify build environment. While pnpm offers faster installs and stricter dependency resolution, its workspace features conflicted with Amplify's build process. The switch to npm resolved build failures with minimal disruption.

9.7 Pivot 7: Cohere Embeddings Back to Self-Hosted Homelab

After migrating to Cohere (Pivot 4), the team encountered aggressive rate limiting on Cohere's free-tier embedding API during Canvas imports, where hundreds of chunks required embedding in rapid succession. Rather than upgrade to a paid tier, the team resolved the underlying Cloudflare Tunnel reliability issues and migrated back to the self-hosted `qwen3-embedding:8b` model on the project lead's homelab. This eliminated rate limits and external API costs entirely. With the tunnel now stable, the team also took the

opportunity to restore the original 4096-dimensional vectors for higher semantic fidelity, accepting the trade-off of sequential pgvector scanning at current data scale. The reranker was similarly moved to self-hosted infrastructure.

Lesson: External API dependencies introduce constraints beyond cost — rate limits can silently degrade user experience. Self-hosting critical pipeline components provides control over throughput at the cost of operational responsibility.

9.8 Budget Constraints

The project operated within an AWS education credits budget. This constraint directly influenced infrastructure decisions: the self-scaling ECS Fargate pattern (scaling to zero when idle) was chosen specifically to minimise compute costs. Instance sizes were kept minimal (`db.t3.micro` for RDS, `cache.t3.micro` for ElastiCache), and services were deployed in a single region (`eu-north-1`) to avoid cross-region transfer costs.

10 Testing

10.1 Testing Approach

The project uses Vitest as its testing framework, configured for the Next.js environment. Testing focused primarily on critical backend logic, including authentication flows, API route handlers, and database query functions.

10.2 What Was Tested

Unit tests cover authentication utilities (password hashing, JWT generation and verification), database query functions (note CRUD, user management), and input validation logic. API route integration tests verify that endpoints return correct status codes and response shapes for both authenticated and unauthenticated requests.

10.3 Honest Assessment

Test coverage is not comprehensive. The frontend components, Canvas import pipeline, and RAG query flow lack automated tests. End-to-end testing was performed manually by the team during development, with each feature verified against its requirements before merging. The team acknowledges that more rigorous automated testing would have caught certain integration issues earlier, particularly around the BullMQ to SQS migration where the failure mode only manifested in the production environment.

11 Project Management

11.1 Methodology

The team followed an agile-inspired approach without strict adherence to any specific framework. Development proceeded in roughly two-week sprints, with priorities set at the beginning of each sprint based on feature dependencies and upcoming deadlines. Communication occurred primarily through GitHub Issues and group messaging.

11.2 GitHub Projects

Task tracking was managed through GitHub Issues and Pull Requests. The repository accumulated 117 issues (96 closed, 21 open) and 54 pull requests over the project lifetime. Issues were categorised by feature area (authentication, editor, Canvas integration, RAG, infrastructure) and assigned to team members. Pull requests enforced code review before merging to the `main` branch, with notable PRs including security hardening batches (PR #155), quiz system implementation (PR #135), and batch issue resolution sprints (PR #133 resolving 37 issues).

Open issues at project close include planned improvements such as centralising RAG pipeline constants, converting worker files to TypeScript, adding CloudWatch metrics for the embedding pipeline, and addressing remaining security advisories — reflecting an honest backlog of known technical debt.

Screenshot: GitHub Issues — Issue tracker showing feature categorisation and resolution

Figure 19: GitHub Issues — Issue tracker showing feature categorisation and resolution

11.3 Timeline

Period	Phase	Key Activities
Sep 2025	Planning	Initial commit, Next.js setup, documentation
Oct–Nov 2025	Semester 1 Dev	Login prototype, auth system, Docker setup
Dec 2025	Semester 1 Wrap	API fixes, deployment debugging, schema work
Jan 2026	Semester 2 Start	Fresh Next.js init, Tailwind templates, restructure
Feb 2026	Core Build	Amplify deployment, S3 storage, editor redesign, auth, RAG, Canvas research
Mar 1–10	Feature Sprint	Canvas import, RAG merge, SQS migration, Fargate, Cohere, chat, i18n
Mar 15–25	Polish & Fix	PDF viewer, DB consolidation, domain migration, SES, translations, UI audit
Mar 26 – Apr 3	Report & Hardening	Documentation, report writing, GDPR account deletion, Marker ASG deployment, ingestion queue refactor, frontend unification

Table 2: Development Timeline

11.4 Commit Activity

The project accumulated over 807 commits over 65 active development days. Activity was heavily concentrated in February and March 2026, with those two months accounting for 688 of the total 807 commits. The most active single day was February 17 with 70 commits. This distribution reflects the semester structure, with planning and prototyping in Semester 1 followed by intensive implementation in Semester 2.

Contributor breakdown (across all branches): Semyon Fox contributed 742 commits, Shreyansh Singh contributed 13 commits, and the remainder comprised automated dependency updates (Dependabot) and minor contributions. The commit distribution reflects the team structure described in Section 2, with Semyon serving as the primary implementer across all subsystems while Samuel and Shreyansh focused on specific feature areas where their contributions were measured more by feature delivery than commit volume.

12 Challenges & Reflections

12.1 Semyon Fox

As technical lead, my biggest challenge was scope management. The combination of AI-assisted development and genuine ambition created a feedback loop: every feature felt achievable, so we kept adding them. The project grew from a simple notes app to a full-stack platform with RAG, Canvas integration, spaced repetition, and a GPU-backed OCR pipeline — all within a university project timeline.

The most technically demanding work was the AWS infrastructure. Debugging why Amplify's Lambda functions couldn't reach ElastiCache (VPC networking), why SQS mes-

sages weren't being consumed (IAM permissions), and why Marker's GPU instance took 15 minutes to cold-start (pip installing marker-pdf on every boot, solved by pre-baking a Docker image) each consumed entire days. These problems don't appear in tutorials or documentation — they emerge from the specific intersection of services in your architecture.

Managing a three-person team while being the primary implementer created an inherent tension. I wanted to empower my teammates to own their contributions, but the interconnected nature of the system meant that integration work often fell to me. In retrospect, I should have invested more time in documentation and interface contracts early on, so that components could be developed more independently.

The key lesson from this project is that architecture decisions made under time pressure have long-lasting consequences. The BullMQ-to-SQS migration, the embedding pipeline iterations (Ollama to Cohere to self-hosted), and the monorepo collapse each cost days of rework. Building a proof-of-concept for each integration point before committing to it would have saved significant time.

12.2 Samuel Regan

For me, the most glaring challenge wasn't writing code. It was the learning curve and the invisible layer between the code and the services.
In a project of this magnitude, there were so many components from the AWS services such as instances and environment variables that when something failed even getting the logs for the deployed project was a huge endeavour. Most of the time the code would fail silently and it was almost always not an internal server error, but a configuration issue. For example, I spent an entire day debugging why the rag pipeline worker queue was not dispatching ingestion requests, and when it did it couldn't find the database or persist any data. The root cause was a second database instance running in parallel. My IDE was pointed to a different, older one, so the SELECT checks always returned empty chunk columns.

The ambition for this project between myself and my teammates compounded this. This was our first chance to develop a real full stack app alongside our course, and thus we had chosen the technologies and methodologies we believed would be most suitable for the project and consequently, I was overwhelmed with a long list of technologies, such as TypeScript, Next.js, and a full array of AWS services, as well as learning team Git workflows for the first time which all formed into a very harsh learning curve.

Upon reflection, I have learnt that on a production level, programming skill and familiarities with the technologies are equally important. For the first half of this project I neglected the services making the production possible and focused on the backend code, and that tunnel vision backfired greatly when I tried to integrate my features into the rest of the system. That said, thanks to this ambitious project I have become far more familiar with cybersecurity, the connection between frontend and backend functions/services such as features being too slow for a UI facing application,integrating LLM models and manipulating them for niche purposes, the services required to host an application on AWS, and the critical thinking required to invent my own solutions to problems, such as our RAG pipeline ingesting too slowly. The very worker queue I described debugging earlier was my solution to that problem, processing documents in an ordered queue with

workers rather than piling on large datasets to be ingested at once.

12.3 Shreyansh Singh

Working on OghmaNotes was my first experience with production cloud infrastructure. Setting up AWS RDS with SSL certificates, configuring S3 bucket policies, and getting SES verified for transactional email each presented their own learning curves. The most challenging aspect was understanding how the various AWS services connect — security groups, IAM roles, and VPC configurations are not intuitive when coming from a local development background.

The OAuth implementation with Auth.js required careful attention to the callback flow and account linking logic, particularly handling edge cases where a user might register with email and later try to sign in with Google using the same email address. The Canvas API integration involved navigating a large REST API surface and understanding the Canvas data model (courses, modules, items, files), which required significant research and experimentation.

The key takeaway from this project was that real-world software engineering is as much about infrastructure and integration as it is about writing application code. The skills I developed in AWS, database design, and API integration are directly applicable to professional work.

13 Future Work

Hybrid Search: Combining vector similarity with BM25 keyword search would improve retrieval for queries containing technical terms or proper nouns that benefit from exact matching alongside semantic understanding.

Collaborative Editing: Real-time collaborative editing would allow study groups to work on shared notes simultaneously, integrated with the RAG system.

Expanded LMS Support: Adding integrations for Moodle, Blackboard, and other learning management systems would broaden the platform's applicability beyond Canvas.

BYOK (Bring Your Own Key): Allowing students to connect their own LLM API keys would provide model choice and reduce platform costs. The settings UI stub already exists.

Offline Support (PWA): Implementing a Progressive Web App with IndexedDB caching would allow students to review notes and flashcards without an internet connection, syncing changes when reconnected.

14 AI-Assisted Development Disclosure

This project used AI coding assistants — primarily Claude Code (Anthropic), with supplementary use of OpenCode and Codex — throughout development. This section describes

the relationship between the developers and these tools.

14.1 How AI Was Used

AI served as an implementation accelerator. The developers defined all architectural decisions, feature priorities, and design constraints; AI tools generated code against those specifications, which was then reviewed, tested, and modified before committing. Common use cases included generating boilerplate for repetitive patterns (API routes, database migrations, i18n translation files), debugging complex AWS configuration issues, and UI refinement.

14.2 What AI Did Not Do

AI did not make architectural decisions. Every significant pivot documented in Section 9 was driven by a constraint the developers encountered at runtime and diagnosed themselves:

- **BullMQ to SQS:** The developer discovered that Amplify's Lambda functions could not reach ElastiCache without VPC configuration. AI did not surface this — the failure occurred in production and was diagnosed through CloudWatch logs and AWS networking documentation.
- **Ollama to Cohere to self-hosted embeddings:** The initial self-hosted embedding pipeline failed under load due to Cloudflare Tunnel instability causing request timeouts. Cohere resolved reliability but introduced rate limiting during bulk Canvas imports. The final pivot back to self-hosted (with tunnel issues resolved) restored full throughput and preserved 4096-dimensional embeddings for higher retrieval fidelity.
- **Marker cold-start fix:** The developer diagnosed that the EC2 GPU instance was pip-installing `marker-pdf` on every boot (causing 15-minute cold starts) and resolved it by pre-baking a Docker image with dependencies installed.

In each case, the developer identified the problem, chose the solution, and directed the implementation. AI accelerated the execution but did not replace the engineering judgement required to navigate these decisions.

14.3 Reflection

AI-assisted development significantly accelerated implementation velocity but introduced a tendency to overscope. The ability to implement features quickly made it tempting to add more than was necessary. The key learning was that AI assistance shifts the bottleneck from "can I implement this?" to "should I implement this now?" — a lesson in scope management that proved as valuable as any technical skill gained during the project.

15 Resources & Links

Resource	URL
Live Application	https://oghmanotes.ie
GitHub Repository	https://github.com/semyonfox/oghma
Canvas LMS API Docs	https://canvas.instructure.com/doc/api/
Cohere Embed API	https://docs.cohere.com/reference/embed
pgvector Documentation	https://github.com/pgvector/pgvector
Auth.js Documentation	https://authjs.dev
AWS Amplify Gen 2	https://docs.amplify.aws
Next.js Documentation	https://nextjs.org/docs

Table 3: Project Resources and References